

TUPLE (interview questions):

1. What is a tuple in Python, and how is it different from a list internally and conceptually?
2. Why are tuples immutable in Python? What advantages does immutability provide?
3. If tuples are immutable, why can a list inside a tuple still be modified? Explain with the concept of references.
4. What is the difference between (10) and (10,)? Why is the comma important?
5. How does Python distinguish between a tuple and an expression enclosed in parentheses?
6. What is tuple packing? Explain with examples.
7. What is tuple unpacking? How does Python perform unpacking internally?
8. What is extended tuple unpacking? Explain the role of the * operator.
9. Can a tuple contain mutable objects? If yes, what happens to tuple immutability in that case?
10. Why can a tuple be used as a dictionary key but a list cannot? Explain using hashability.
11. What is the relationship between tuple immutability and hashability? Are all tuples hashable? Explain.
12. Why is (1, [2, 3]) generally not hashable even though the outer object is a tuple?
13. What is the difference between shallow immutability and deep immutability in tuples?
14. Why does Python provide only a small number of tuple methods compared to list methods?
15. Explain the difference between tuple.count() and tuple.index().
16. What happens when tuple.index() cannot find the requested element? Why?
17. How does tuple comparison work in Python? Explain lexicographical comparison.
18. Can tuples contain elements of different data types? Why does Python allow heterogeneous tuples?
19. What is the difference between a tuple containing one element and a tuple containing multiple elements?
20. Can you change an element of a tuple indirectly? Explain the difference between modifying an element and replacing an element.
21. Why is tuple concatenation considered to create a new tuple rather than modify the existing tuple?
22. What is tuple repetition? Why does multiplying a tuple by an integer not multiply its individual values?
23. What happens when a tuple is multiplied by 0 or a negative integer? Explain conceptually.
24. What is the difference between a = (1, 2) and a = 1, 2?
25. How does tuple unpacking help in swapping two variables without using a temporary variable? Explain the mechanism.

26. What are the advantages of using tuples instead of lists in real-world Python programs?
27. Why are tuples generally more memory-efficient than lists? Explain at a high level.
28. Why can tuples sometimes be faster than lists for iteration and access?
29. What is the difference between a tuple and a named tuple? When would you prefer namedtuple or similar structured alternatives?
30. Explain the role of tuples in function arguments and return values. How can a function return multiple values using a tuple?

CODING QUESTIONS:

1. What will be the output?

```
t = (10, 20, 30)
```

```
t[1] = 50
```

```
print(t)
```

Why does this happen?

2. What is the difference between these two?

```
a = (10) b =
```

```
(10,)
```

```
print(type(a))
```

```
print(type(b))
```

```
)
```

3. Predict the output:

```
t = (1, 2, [3, 4]) t[2].append(5)
```

```
print(t)
```

How can a tuple be modified if tuples are immutable?

4. Predict the output:

```
a = (1, 2, 3) b
```

```
= (1, 2, 3)
```

```
print(a is b)
```

```
print(a == b)
```

5. What will happen?

```
t = (1, 2, 3) print(t  
* 2)
```

6. What will be the output?

```
t = (1, 2, 3, 2, 2)  
print(t.count(2))  
print(t.index(2)) 7.
```

Predict the output:

```
t = (1, 2, 3) a,  
b, c = t  
print(a, b, c)
```

8. What happens here?

```
t = (1, 2, 3) a, *b = t  
print(a) print(b)  
print(type(b)) 9.
```

Predict the output:

```
t = (1, 2, 3, 4, 5) a,  
*b, c = t print(a,  
b, c)
```

10. What happens here?

```
t = (1, 2, 3) a,  
b = t
```

Explain the error.